

Détection de malwares à l'ère des menaces génératives en environnement SOC

Etudiant :
GOHAUT Renan

Mastère Spécialisé :
Expert Forensic et Cybersécurité

Responsable Pédagogique :
ELGALAI Reza

Année Scolaire :
2025 - 2026

Résumé (150 mots)

Ce mémoire rend compte d'un travail mené au cours de mon alternance chez Login Sécurité, prestataire de services de sécurité managés (MSSP) du groupe Constellation, où j'ai occupé le poste d'analyste SOC/VOC.

L'étude n'était pas au programme. Je l'ai engagée en marge de mes activités, sur un échantillon dont la classe méritait selon moi une caractérisation complète, et les règles qui en sortent peuvent alimenter la base de détection du parc client si l'équipe les valide.

Le travail s'est déroulé en trois temps :

- analyse statique du binaire, pour en extraire les caractéristiques structurelles ;
- analyse dynamique de son comportement en environnement confiné ;
- traduction de ces observations en règles YARA et Sigma, puis en plugin Volatility.

Un SOC qui attend la signature de l'éditeur ne détecte que ce qui est déjà répertorié. Entre la découverte d'une menace et sa prise en charge par les produits du marché, il s'écoule des jours. Savoir caractériser un binaire soi-même, c'est les récupérer.

Entreprise : Login Sécurité

Lieu : *199 Bureaux de la Colline, 92210 Saint-Cloud*

Responsable : Jean-Christophe PILLET

Mots clés (cf Thésaurus)

- Rétro-ingénierie
- Analyse de logiciels malveillants
- Détection d'intrusion
- Security Operation Center (SOC)

Remerciements

Christophe Rieunier a fait la route depuis Nantes pour nous parler d'analyse de *malware*. Il aurait pu dérouler son support et repartir. Au lieu de ça, il est resté le soir pour nous montrer les conférences qu'il avait données au fil de sa carrière, et pour raconter ce qu'il y avait vu. On est restés aussi. Pas parce qu'on suivait tout, loin de là, mais parce qu'on n'avait pas envie que ça s'arrête. Il y a des gens qui vous transmettent leur passion sans jamais chercher à impressionner : il est de ceux-là. Si ce mémoire parle de *malwares*, c'est grâce à vous. Alors merci.

Merci à Reza El Galai, responsable pédagogique du Mastère spécialisé, d'avoir tenu la promo à distance de Troyes sans jamais nous lâcher. Encadrer des gens qu'on ne croise pas dans un couloir demande un effort qu'on ne remarque que le jour où il n'est pas fait. Et c'est lui qui est allé chercher des intervenants comme Christophe : un cours vaut d'abord par celui qui le donne, et ces choix-là ne se font pas tout seuls.

Merci à Jean-Christophe Pillet de m'avoir pris comme alternant chez Login Sécurité. Faire confiance à quelqu'un qui débute, ça coûte du temps, et il en a donné sans compter, disponible pour mes questions, y compris celles dont je savais en les posant qu'elles étaient bêtes. J'ai autant appris en travaillant dans son équipe qu'en suivant des cours. C'est une chance que j'aimerais avoir la lucidité de mesurer encore dans dix ans.

Merci enfin à ma famille, qui ne m'a pas regardé de travers quand j'ai annoncé que je repartais en études. Et merci surtout à ma compagne, Mathilde, qui a tenu trois ans de relation à distance et qui a relu ces pages. Trente pages sur la détection de *malware* quand on n'est pas du métier, c'est une preuve d'amour. Elle a tenu bon, donc moi aussi.

Sommaire

Remerciements

Liste des tableaux

Table des figures

Introduction	1
1 État de l'art de la menace logicielle et mécanismes d'évasion	2
Périmètre	2
1.1 Définition et taxonomie du malware moderne	2
1.2 Classification par objectifs opérationnels	4
1.3 Évasion statique : rendre le binaire illisible	6
1.4 Évasion à l'exécution : échapper à l'observation	7
1.5 Intégration de modèles de langage dans la chaîne offensive	10
1.6 État de l'art de la détection et de ses limites	12
1.7 Synthèse et verrou identifié	13
2 Étude de cas : analyse de PromptLock	15
2.1 Présentation du cas	15
2.2 Analyse technique de l'échantillon	16
2.3 Outils de détection développés	22
Conclusion	30
Bibliographie	
A Annexes	I
A.1 Captures d'écran	I
A.2 Formules chimiques	I
A.3 Extraits de code source	I

Liste des tableaux

1.1	Lecture de l'identifiant <code>Trojan:Linux/XorDDoS.A!ml</code>	3
1.2	Les cinq niveaux d'analyse d'une intrusion	4
1.3	Techniques d'évasion statique observées sur ELF	7
2.1	Indices de compromission identifiés lors de l'analyse statique de PromptLock . .	25
2.2	Indicateurs réseau complémentaires confirmés lors de l'analyse dynamique	26
2.3	Cartographie des techniques observées ou déduites selon le référentiel MITRE ATT&CK	27
2.4	Techniques MITRE ATLAS applicables à PromptLock	28
2.5	Analyse comparative ATT&CK / ATLAS appliquée à PromptLock	29

Table des figures

2.1	Chaîne d'exécution de PromptLock, de <code>main.main</code> jusqu'au chiffrement local conditionnel.	16
2.2	Graphe d'appel reconstruit à l'aide de GoReSym et Ghidra, annoté selon le niveau de preuve associé à chaque fonction.	19

Introduction

L'institut AV-TEST recense chaque jour plus de 450 000 nouveaux logiciels malveillants [6]. Le chiffre impressionne moins qu'il n'y paraît. Ces échantillons ne sont pas 450 000 menaces distinctes : ce sont, pour l'essentiel, des variantes produites par des moteurs de mutation dont la logique demeure finie. Un analyste qui reconstitue cette logique neutralise la famille entière. La détection par signature tient sur ce pari.

Le rapport *AI Threat Tracker* publié le 5 novembre 2025 par le Google Threat Intelligence Group documente les premiers programmes qui ne le tiennent plus [7]. GTIG y décrit des *malwares* qui interrogent un modèle de langage pendant leur exécution, pour générer des scripts, obscurcir leur propre code ou produire des fonctions malveillantes à la demande. PROMPTSTEAL, employé contre des cibles ukrainiennes, interroge un modèle hébergé sur Hugging Face pour générer ses commandes Windows au lieu de les coder en dur. PROMPTFLUX sollicite l'API Gemini pour se réécrire « *just-in-time* ». PROMPTLOCK, rançongiciel, génère à l'exécution les scripts Lua qui parcourent le système de fichiers, sélectionnent les données et les chiffrent. L'intelligence artificielle générative n'intervient plus seulement en amont, dans la rédaction de courriels de *phishing* ou l'écriture assistée de charges utiles : elle est appelée par le programme lui-même, sur la machine de la victime.

Jusqu'ici, le polymorphisme et le métamorphisme restaient déterministes. Le code mutait, mais selon des règles finies, qu'un analyste pouvait reconstituer et généraliser. C'est ce postulat que la génération à l'exécution abolit. Il n'y a plus de moteur de mutation à rétro-concevoir, seulement une invite et un modèle dont les sorties varient d'une itération à l'autre.

Encore faut-il le vérifier ce comportement plutôt que lire ces rapports. Ce mémoire reprend le dernier de ces trois échantillons et l'analyse statiquement, puis dynamiquement, avant de convertir les observations en règles YARA, en règles Sigma et en *plugin* Volatility. Ce parcours, du binaire à la règle, est celui que suit un analyste en SOC lorsqu'un échantillon inconnu lui parvient.

Il conduit à la question qui structure ce travail. Nos mécanismes de détection supposent un adversaire dont le comportement demeure *in fine* modélisable. Que valent-ils face à un code qui change à chaque itération et s'optimise contre les défenses qu'il rencontre ?

Chapitre 1

État de l'art de la menace logicielle et mécanismes d'évasion

Périmètre

La souche analysée dans la suite de ce mémoire est un binaire ELF destiné à un environnement GNU/Linux. Cet état de l'art est donc construit autour des systèmes Linux, en particulier dans leurs usages serveur et cloud, et ne mobilise l'écosystème Windows qu'à titre de contrepoint lorsque la comparaison éclaire un mécanisme.

Ce choix n'est pas seulement dicté par l'objet d'étude. La littérature disponible est très déséquilibrée : les travaux sur la détection de binaires PE forment un corpus mature, doté de jeux de données publics et de protocoles d'évaluation partagés, tandis que la recherche équivalente sur les binaires ELF reste dispersée et beaucoup moins outillée [14]. Ce déséquilibre a une conséquence directe sur le travail présenté ici, puisqu'il interdit de reprendre telles quelles les méthodes d'évaluation du monde Windows, et il constitue en lui-même une partie de la justification du sujet.

Les sources mobilisées relèvent de trois registres qui n'ont pas la même valeur probante : travaux académiques évalués par les pairs, rapports d'investigation publiés par des équipes de réponse à incident, et documentation de référence des cadres d'analyse. Le texte signale explicitement les cas où une capacité relève de l'hypothèse plutôt que de l'observation.

1.1 Définition et taxonomie du malware moderne

Un malware ne se définit plus par sa capacité de réplication, comme pouvaient l'être les premiers virus des années 1980. Ce qui le caractérise est l'intention : tout code conçu pour s'introduire dans un système, en perturber le fonctionnement, en extraire des données ou y maintenir un accès non autorisé, sans le consentement de son propriétaire.

Le critère est bien l'intention, et non la technique employée. Un outil de prise en main à distance est un logiciel d'administration parfaitement légitime quand un exploitant l'installe sur ses serveurs, et une porte dérobée quand un attaquant l'y dépose. Les deux peuvent être le même programme. Cette ambiguïté traverse tout ce chapitre : elle explique pourquoi la détection est difficile, et pourquoi elle ne se ramène jamais à reconnaître un objet dans une liste.

Le déplacement de définition accompagne un changement de nature de l'activité. La production de code malveillant, longtemps portée par des motivations de démonstration technique, s'est structurée en filière, avec une division du travail et des objectifs financiers ou stratégiques. Le volume de nouveaux échantillons recensés quotidiennement par les observatoires du secteur donne la mesure de cette industrialisation [6].

1.1.1 Nomenclature et standards d'identification

Deux analystes qui examinent le même fichier dans deux entreprises différentes doivent pouvoir établir qu'ils parlent bien du même objet. C'est le rôle des conventions de nommage, dont la plus ancienne et la plus diffusée est celle du CARO, définie au début des années 1990 et que Microsoft déclare suivre pour ses propres détections [11]. Un identifiant y est composé de cinq champs, à la manière d'un nom d'espèce en biologie, du plus général au plus précis :

Type : Plateforme / Famille . Variante ! Suffixe

Le tableau 1.1 décompose un identifiant réel selon ce schéma.

Champ	Valeur	Signification
Type	Trojan	Ce que le code fait : il se présente comme légitime pour être exécuté. Autres valeurs courantes : Backdoor (accès distant), Ransom (rançongiciel), Worm (propagation autonome), PWS (vol de mots de passe).
Plateforme	Linux	L'environnement visé. Peut aussi désigner un langage interprété (Python, Shell) ou une version de Windows.
Famille	XorDDoS	Le nom donné à la découverte, partagé par tous les échantillons dont le code est jugé assez proche pour supposer une origine commune.
Variante	A	Une lettre attribuée dans l'ordre des versions successives : .AF succède à .AE.
Suffixe	!ml	Une précision technique facultative. Ici, la détection provient d'un modèle d'apprentissage automatique et non d'une règle écrite à la main.

TABLE 1.1 – Lecture de l'identifiant Trojan:Linux/XorDDoS.A!ml

Sur les cibles Linux, le champ plateforme masque une difficulté propre au reste de ce mémoire. Un même code source y est fréquemment compilé pour plusieurs architectures matérielles, x86-64 pour les serveurs, ARM ou MIPS pour les équipements embarqués. Les binaires produits sont différents alors que le comportement est identique, et le champ plateforme n'en rend pas compte.

Les limites du schéma.

Présenter cette convention comme un standard sectoriel serait inexact. L'un de ses auteurs reconnaît qu'aucun produit n'en respecte intégralement les règles et que le CARO n'a jamais disposé de l'autorité nécessaire pour les imposer [2]. Deux éditeurs attribuent donc régulièrement des noms différents au même échantillon, et les tentatives ultérieures d'unification ont toutes échoué.

Le problème n'est pas seulement documentaire. Entraîner un modèle à reconnaître des familles suppose de disposer d'échantillons correctement étiquetés ; or l'étiquette de référence, en pratique, est le verdict des antivirus. Si ces verdicts se contredisent, les étiquettes sont bruitées,

et le modèle apprend en partie ce désaccord. Des outils de consolidation existent, mais ils héritent des divergences qu'ils agrègent.

1.1.2 Tactique, technique, vecteur, capacité et charge utile

Cinq notions se confondent couramment, et leur confusion fausse l'analyse de risque : on croit se protéger d'une menace quand on ne se protège que d'un de ses outils. Le cadre MITRE ATT&CK [12] fournit une grille qui les sépare. Une comparaison avec un cambriolage en rend la logique immédiate, et le tableau 2.3 la transpose sur un cas informatique.

Niveau	Question posée	Cambriolage	Intrusion sur un serveur Linux
Tactique	Que cherche l'attaquant à cette étape ?	Entrer dans la maison	Obtenir un premier accès (TA0001)
Technique	Comment s'y prend-il ?	Crocheter la serrure	Exploiter une application exposée (T1190)
Vecteur	Par quel support concret ?	La porte de service	Un service web accessible depuis internet
Capacité	Que sait-il faire une fois entré ?	Ouvrir un coffre	Élever ses privilèges, se dissimuler
Charge utile	Qu'emporte-t-il au final ?	Les bijoux	Chiffrer les données, exfiltrer des clés

TABLE 1.2 – Les cinq niveaux d'analyse d'une intrusion

Une sixième notion traverse les autres : l'évasion défensive (TA0005), qui regroupe tout ce que l'attaquant met en œuvre pour ne pas être vu pendant qu'il agit. Elle constitue l'objet principal de ce chapitre, et son cambrioleur porterait des gants.

L'intérêt de cette grille est pratique. Une signature protège d'un échantillon, c'est-à-dire d'un cambrioleur précis dont on possède la photographie ; une détection comportementale protège d'une technique, donc de quiconque crochète une serrure. La première approche est exacte mais s'échappe dès que l'apparence change, la seconde tolère la variation mais produit des faux positifs, puisque le propriétaire aussi manipule sa serrure. Tout l'état de l'art de la détection tient dans cet arbitrage, et la section 1.6 y revient.

1.2 Classification par objectifs opérationnels

Classer les menaces par l'objectif de leurs auteurs est plus utile en pratique qu'une taxonomie par type de fichier.

Le rançongiciel chiffre les données et monnaie leur restitution. Sur Linux, il a suivi la valeur : les variantes ELF visant les hyperviseurs de virtualisation permettent de rendre indisponibles des dizaines de machines virtuelles par une seule exécution, ce qui déplace le rapport coût-bénéfice en faveur de l'attaquant.

Le voleur d'informations extrait discrètement des données valorisables. Sur un poste de travail il vise des cookies et des identifiants ; sur un serveur ou une machine de développement, il vise des secrets d'infrastructure, jetons d'API, clés SSH, variables d'environnement d'intégration

continue, dont la compromission ouvre un accès bien plus large que la machine elle-même.

La porte dérobée assure un accès persistant à distance. C'est la catégorie la plus représentée dans les compromissions de serveurs, et celle qui concentre l'essentiel des techniques de dissimulation étudiées en section 1.4.

Le mineur de cryptomonnaie détourne les ressources de calcul. Longtemps considéré comme une nuisance mineure, il sert en pratique de révélateur : sa présence signale une chaîne d'accès qui aurait tout aussi bien pu porter une charge destructive.

Le bot enrôle la machine dans un réseau contrôlé à distance. Les familles visant les équipements embarqués sous Linux, dérivées pour beaucoup d'un même code source diffusé publiquement, illustrent le problème de la compilation croisée : un même comportement se décline en dizaines de binaires dont la structure varie selon l'architecture cible.

Ces catégories ne sont pas étanches, et un échantillon combine ordinairement plusieurs fonctions.

1.2.1 L'écosystème économique

La spécialisation des acteurs explique une part de cette polyvalence. Le modèle du rançongiciel comme service sépare ceux qui écrivent le code de ceux qui conduisent les intrusions. En amont, des courtiers revendent des accès déjà obtenus sans participer à l'exploitation ; l'ANSSI en fait des intermédiaires devenus indispensables à l'écosystème, en particulier pour les rançongiciels [1].

Le mode opératoire Houken l'illustre. L'exploitation des vulnérabilités initiales y est nettement plus soignée que la suite de la compromission, écart qui trahit le passage de relais entre un courtier et les acteurs venus après lui. Autrement dit, la charge finale ne renseigne plus sur le niveau technique de celui qui a ouvert la porte.

Les volumes du CERT-FR donnent l'échelle. L'agence a connu 1 366 incidents sur l'exercice 2025, contre 831 en 2022. Le rançongiciel recule légèrement, à 128 compromissions, et aucune souche ne domine : Qilin pèse 21 % des cas, Akira 9 %, LockBit 5 %, et plus d'une dizaine de souches inédites apparaissent sur au moins un incident dans l'année. L'exfiltration de données, elle, passe de 130 à 196 incidents[1].

Le déplacement le plus gênant pour la détection est ailleurs. Plusieurs courtiers en accès, dont TA577, TA571 et TA544, ont abandonné les chargeurs et les botnets au profit d'outils d'administration à distance légitimes et de binaires natifs du système. Lumma Stealer, de son côté, renouvelle ses domaines de commande et contrôle plusieurs fois par semaine, un même échantillon pouvant en embarquer une dizaine.

Deux pressions pèsent donc sur une défense construite sur des artefacts figés. La fragmentation d'abord : sans souche dominante, il faut couvrir une longue traîne de familles pour un rendement décroissant. Le reste est plus radical. Quand l'attaquant se sert d'un outil d'administration signé plutôt que d'un programme malveillant, il n'y a plus d'artefact à reconnaître.

1.3 Évasion statique : rendre le binaire illisible

L'analyse statique consiste à examiner un fichier sans l'exécuter. Les techniques qui la contredisent forment la première ligne d'évasion.

1.3.1 Réduction de la surface d'analyse

Sur ELF, les mesures les plus efficaces sont souvent les plus élémentaires. La suppression des symboles prive l'analyste des noms de fonctions et de variables. La liaison statique fusionne les bibliothèques dans le binaire et supprime la table d'importation, qui constitue par ailleurs l'un des indicateurs les plus utilisés pour caractériser un exécutable. Le recours à des langages compilés modernes, Go ou Rust, produit des binaires volumineux dont l'exécution est dominée par le code de la bibliothèque standard, ce qui noie la logique métier dans un bruit considérable sans qu'aucune intention d'obfuscation ait été nécessaire.

L'altération volontaire des en-têtes et des tables de sections complète l'ensemble : le chargeur du noyau tolère des structures que les outils d'analyse refusent, et cet écart de tolérance suffit à mettre en échec un outillage automatisé.

1.3.2 Empaquetage

Un packer compresse ou chiffre l'exécutable original et lui adjoint une routine de déballage exécutée au lancement. Sur ELF, l'usage d'un packer public dont les marqueurs ont été modifiés reste très répandu, précisément parce qu'il neutralise les outils de déballage automatique sans exiger de développement propre.

Une taxonomie des structures de déballage a été établie sur le monde PE, distinguant les modèles à couche unique des schémas incrémentaux où le code original n'est jamais entièrement reconstitué en mémoire à un instant donné [18]. Cette dernière catégorie invalide l'hypothèse implicite de nombreuses chaînes d'analyse automatisée, celle d'un instant de capture où le code complet serait disponible.

1.3.3 Polymorphisme, métamorphisme et virtualisation

Le polymorphisme chiffre le corps du code et fait varier la clé à chaque exécution : l'empreinte sur disque diffère, le contenu déchiffré en mémoire reste identique. Le métamorphisme va plus loin en réécrivant le code lui-même à chaque infection, par substitution d'instructions équivalentes, permutation de registres ou dispersion en fragments reliés par des sauts [17].

Ces moteurs ont été développés et documentés dans le contexte des virus DOS et Windows. Ils sont nettement plus rares sur ELF, pour une raison économique : maintenir un moteur de réécriture correct sur plusieurs jeux d'instructions coûte cher, alors que l'écosystème Linux offre des voies d'évasion moins onéreuses, décrites en section 1.4. L'observation mérite d'être retenue, car elle indique où porter l'effort défensif.

La virtualisation de code, qui traduit les instructions en un bytecode propriétaire interprété par une machine virtuelle embarquée, et les prédicats opaques, conditions dont le

résultat est connu de l'auteur mais indécidable pour un désassembleur [3], complètent l'arsenal. Ces deux techniques restent surtout associées aux protecteurs commerciaux.

Technique	Principe	Fréquence sur ELF
Suppression des symboles	Retrait des noms de fonctions et variables	Très courante
Liaison statique	Suppression de la table d'importation	Très courante
En-têtes altérés	Structures tolérées par le noyau, refusées par l'outillage	Courante
Empaquetage	Compression ou chiffrement, déballage au lancement	Courante
Déballage incrémental	Reconstitution partielle et successive en mémoire	Peu fréquente
Polymorphisme	Chiffrement du corps, clé variable	Peu fréquente
Métamorphisme	Réécriture du code à chaque infection	Rare (coût multi-architectures)
Virtualisation	Bytecode propriétaire et interpréteur embarqué	Rare

TABLE 1.3 – Techniques d'évasion statique observées sur ELF

1.4 Évasion à l'exécution : échapper à l'observation

L'obfuscation traite le problème de la lecture du code. Elle ne dit rien du second front, celui de l'exécution sous surveillance, où se joue l'essentiel des compromissions de serveurs. Un code parfaitement obfusqué qui déclenche un chiffrement massif sera arrêté ; un code trivial qui n'émet aucun signal observable ne le sera pas.

1.4.1 Anti-analyse et conditionnement de l'environnement

Avant de déployer sa charge, un code cherche à établir s'il s'exécute sur une cible réelle ou dans un environnement d'analyse. Sous Linux, les vérifications les plus simples passent par le pseudo-système de fichiers `/proc`, qui expose l'état du processus, y compris la présence d'un débogueur attaché. L'auto-attachement par `ptrace` produit le même résultat par un autre chemin : un processus ne pouvant être tracé qu'une fois, réussir à se tracer soi-même prouve l'absence d'observateur.

D'autres contrôles portent sur la nature de l'hôte, avec la recherche d'indices de virtualisation ou de conteneurisation, sur la temporisation, une exécution instrumentée étant plus lente d'un facteur mesurable, ou sur des caractéristiques d'environnement propres à la cible visée. Le conditionnement environnemental (T1480.001) pousse la logique jusqu'au bout : la clé de déchiffrement est dérivée de propriétés de la machine ciblée, de sorte que le code ne se déchiffre nulle part ailleurs, y compris entre les mains du défenseur.

1.4.2 Exécution sans fichier

L'analyse statique suppose un fichier à analyser. L'appel système `memfd_create` crée un objet mémoire anonyme utilisable comme descripteur exécutable, ce qui permet de charger et d'exécuter un binaire sans jamais l'écrire sur le système de fichiers. Le montage en mémoire vive de `/dev/shm` offre une variante moins discrète mais tout aussi accessible.

Le corollaire est important pour la démarche défensive : sur un hôte ainsi compromis, il n'existe pas d'échantillon à collecter par les moyens habituels, et la détection doit reposer entièrement sur la télémétrie d'exécution.

1.4.3 Dissimulation par rootkit : trois couches

Les rootkits Linux se répartissent selon la couche où ils s'insèrent, chacune offrant un compromis différent entre furtivité et fragilité [10].

La couche utilisateur repose sur le détournement de l'éditeur de liens dynamique. En déclarant une bibliothèque partagée dans `/etc/ld.so.preload`, l'attaquant impose que ses propres implémentations soient résolues avant celles de la bibliothèque C standard. Les fonctions de parcours de répertoires ou d'énumération de processus renvoient alors des résultats filtrés, et les outils d'administration usuels deviennent aveugles. La technique est ancienne et triviale à mettre en œuvre ; elle reste efficace contre un défenseur qui interroge le système avec les outils du système.

La couche noyau par module chargeable offre une furtivité supérieure au prix d'une contrainte forte : le module doit être compatible avec le noyau cible, et les mécanismes de signature en compliquent le chargement.

La couche eBPF constitue le développement le plus notable des dernières années. Conçue pour permettre l'exécution de programmes vérifiés dans le noyau, cette infrastructure sert désormais aussi à y accrocher des routines de dissimulation. Un rootkit peut intercepter les appels d'énumération de répertoires pour masquer fichiers et processus, et intercepter l'interface de gestion eBPF elle-même pour se soustraire aux outils censés inventorier les programmes chargés.

Un cas instructif.

L'analyse d'une porte dérobée découverte sur une infrastructure cloud compromise, publiée par une équipe de réponse à incident française, documente une architecture en deux modules eBPF, l'un dédié à la dissimulation, l'autre à l'activation du canal de commande sur réception d'un paquet réseau caractéristique [16]. Le point remarquable tient au mécanisme de repli : lorsque la configuration du noyau ne permet pas le fonctionnement des accroches eBPF, le code bascule automatiquement sur la technique de préchargement de bibliothèque décrite plus haut.

Ce comportement mérite d'être souligné, car il documente une capacité d'adaptation à l'environnement d'exécution, obtenue sans aucun recours à l'intelligence artificielle, par une simple logique de repli programmée à l'avance. On y reviendra en section 1.5.

1.4.4 Neutralisation de la télémétrie

Un agent de sécurité ne voit que ce que le système lui expose. Sous Linux, la télémétrie d'exécution provient principalement du sous-système d'audit du noyau et, pour les outils récents, de sondes eBPF placées sur les points d'entrée des appels système.

Cette dépendance crée une vulnérabilité structurelle. Des travaux récents montrent qu'un code disposant des privilèges suffisants peut agir sur le flux d'événements lui-même, en filtrant ou en supprimant des enregistrements avant qu'ils n'atteignent le tampon lu par l'outil de sécurité. L'agent continue de fonctionner, ses tableaux de bord restent alimentés, et son silence sur une activité donnée est interprété comme une absence d'activité.

La difficulté est ici de nature différente de celle posée par l'obfuscation. Elle ne concerne pas la capacité d'analyse mais la fiabilité de la source, et aucune amélioration du moteur de détection ne la corrige.

1.4.5 Persistance et mimétisme

Les mécanismes de persistance sous Linux sont nombreux et légitimes, ce qui les rend difficiles à surveiller sans générer de bruit : unités `systemd`, tâches planifiées, fichiers d'initialisation de shell, préchargement de bibliothèques.

Le mimétisme s'y ajoute. Un service malveillant nommé de manière à évoquer un composant système standard, installé dans une arborescence plausible, ne se distingue d'un service légitime par aucune propriété observable en dehors de son comportement. Le cas cité plus haut recourait précisément à ce procédé. La question n'est donc pas de détecter une anomalie, mais de discriminer entre deux objets dont la seule différence est l'intention.

1.4.6 Commande, contrôle et exfiltration

Le canal de commande reste le point le plus exposé, et les techniques de dissimulation y visent toutes à réduire la fenêtre d'observation.

L'activation sur paquet caractéristique, dont le cas analysé plus haut fournit une illustration, supprime toute activité réseau tant que le déclencheur n'a pas été reçu : la porte dérobée n'ouvre son port qu'après signal, ce qui la rend invisible à toute cartographie de services. L'usage de services web légitimes comme relais place le trafic dans des flux qu'une organisation ne peut pas bloquer sans coût opérationnel. La génération algorithmique de domaines rend le blocage par liste impraticable.

Une remarque prépare la section suivante : le trafic vers les interfaces de programmation de modèles de langage réunit désormais les propriétés qui font un bon canal de dissimulation, à savoir un volume chiffré, des destinations réputées et un usage professionnel banalisé.

1.4.7 Contrepoint : les spécificités Windows

Sur Windows, l'évasion à l'exécution s'organise autour de trois surfaces d'observation dont l'équivalent n'existe pas sous Linux. L'interface d'analyse antimalware, qui permet l'inspection

des contenus scriptés avant exécution, le mécanisme de traçage d'événements, qui alimente la télémétrie des agents, et l'interception en espace utilisateur, que les agents mettent en place dans les bibliothèques système et que le code malveillant cherche à défaire.

La comparaison éclaire une différence de fond. Sur Windows, l'observation repose largement sur des points d'accroche installés en espace utilisateur, que le code observé peut atteindre. Sous Linux, elle repose sur des primitives noyau, ce qui élève le niveau de privilège requis pour la neutraliser, mais concentre toute la confiance sur une infrastructure unique dont la compromission est d'autant plus lourde de conséquences.

1.5 Intégration de modèles de langage dans la chaîne offensive

L'usage de modèles de langage pendant l'exécution d'un code malveillant a été largement commenté depuis 2025. Cette section examine ce que les cas documentés établissent, en séparant les observations des projections.

1.5.1 Les cas documentés

Un inventaire publié fin 2025 recense les premiers échantillons interrogeant un modèle pendant leur exécution [7]. Quatre logiques distinctes s'y dégagent, que la vulgarisation confond souvent.

La première est l'auto-réécriture : un chargeur écrit en VBScript interroge périodiquement une interface de modèle pour obtenir de nouvelles variantes obfusquées de son propre code source. C'est le seul cas relevant strictement de cette catégorie, et il vise Windows.

La deuxième est la génération d'instructions à la demande : un outil de collecte attribué à un acteur étatique interroge un modèle hébergé pour produire les commandes à exécuter, sans que son propre code soit modifié.

La troisième détourne les ressources de l'hôte : un voleur de jetons de développement exploite les outils d'IA en ligne de commande déjà installés sur la machine pour rechercher d'autres secrets. Le modèle n'est ici ni embarqué ni distant, mais emprunté.

La quatrième, la plus pertinente pour un environnement Linux, concerne un rançongiciel écrit en Go qui interroge un modèle hébergé localement pour produire à la volée les scripts d'énumération, d'exfiltration et de chiffrement. Présenté d'abord comme le premier rançongiciel assisté par IA [15], il s'est avéré être le prototype d'un travail académique [4]. Son intérêt tient au modèle local : la détection fondée sur l'observation d'un trafic vers une interface distante y est sans effet, et rien n'interdit ce schéma sur un serveur disposant de ressources de calcul.

1.5.2 Ce que ces cas établissent, et ce qu'ils n'établissent pas

L'évaluation doit rester mesurée, sous peine de fragiliser le raisonnement défensif qu'elle fonde.

Le chargeur auto-réécrivant est évalué par ses découvreurs comme relevant du développement ou du test, sans capacité de compromettre une cible, et son accès à l'interface du modèle a

été révoqué [7]. Le rançongiciel à modèle local n'a jamais été observé en opération. Plusieurs de ces échantillons étaient détectés par une part significative des moteurs du marché au moment de leur publication.

Ces cas n'établissent donc pas l'existence d'une génération opérationnelle de codes autonomes. Ils établissent trois points plus modestes et plus solides. La faisabilité technique de l'intégration d'un modèle dans la boucle d'exécution est démontrée sur quatre logiques distinctes. Le recours à un modèle local supprime l'indicateur réseau qui constituait la contre-mesure la plus immédiate. Enfin, la trajectoire est ascendante : un rapport ultérieur du même groupe fait état d'un exploit de vulnérabilité inédite dont les analystes estiment avec un haut degré de confiance qu'il a été produit avec l'aide d'un modèle [8].

C'est cette trajectoire, et non l'état des échantillons actuels, qui justifie l'attention défensive.

1.5.3 L'injection de prompt comme technique d'évasion

Un cas du même inventaire inverse la relation entre modèle et code malveillant. Un shell inversé écrit en PowerShell y intègre des instructions en langage naturel destinées non pas à un modèle utilisé par l'attaquant, mais aux dispositifs d'analyse pilotés par modèle susceptibles d'examiner l'échantillon, auxquels il enjoint de le déclarer inoffensif [7].

L'intérêt du cas dépasse sa sophistication, qui est faible. Il montre que confier à un modèle l'évaluation d'un contenu non fiable revient à hériter du problème, non résolu, de l'injection de prompt indirecte. Le point vaut pour tout dispositif de triage qui délègue l'examen d'un fichier suspect à un modèle. Il vaut aussi pour l'analyste : les prompts embarqués dans l'échantillon étudié au chapitre 2 sont du texte écrit par l'attaquant, et rien n'oblige leur auteur à les rédiger à l'intention du seul modèle qu'il interroge.

1.5.4 L'adaptation contextuelle : une capacité déjà présente

La capacité qui inquiéterait le plus n'est pas la mutation de forme mais l'adaptation à l'environnement : un code qui reconnaîtrait les dispositifs de sécurité en place et ajusterait sa charge en conséquence.

Aucun des échantillons assistés par modèle documentés à ce jour ne présente cette capacité. Mais elle n'est pas hypothétique pour autant, et c'est là que le point de vue Linux change l'analyse : le mécanisme de repli du rootkit eBPF décrit en section 1.4 en est déjà une forme aboutie [16]. Le code y évalue la configuration du noyau et choisit sa technique de dissimulation en conséquence, par une logique de repli programmée à l'avance.

L'adaptation contextuelle est donc une capacité opérationnelle attestée, obtenue sans modèle de langage. Ce que l'intégration d'un modèle changerait n'est pas sa nature mais son étendue : le passage d'un arbre de décision fini, écrit par un développeur et donc énumérable par un analyste, à une génération ouverte dont les branches ne sont pas connues à l'avance. Formulée ainsi, l'hypothèse devient testable, ce qui n'était pas le cas de sa version initiale.

1.6 État de l'art de la détection et de ses limites

Quatre approches coexistent, dont aucune ne suffit isolément.

1.6.1 Détection statique par indicateurs

L'approche ne se réduit pas au condensat cryptographique du fichier. Elle mobilise également des signatures de motifs d'octets, des règles combinant chaînes et conditions structurelles, des empreintes calculées sur des sections ou des tables du binaire, et des fonctions de hachage approché tolérantes aux modifications mineures.

Cette gradation nuance la critique habituelle. Un condensat exact est effectivement mis en échec par la moindre mutation, mais une règle portant sur une routine de déballage ou un hachage approché résistent à une partie du polymorphisme. Ce qui met réellement l'approche en difficulté est l'effet de volume : le coût marginal de production d'une variante inédite est aujourd'hui inférieur au coût marginal de production d'une règle.

Sur ELF, une difficulté supplémentaire tient à la compilation croisée. Une règle validée sur une compilation x86-64 ne se transpose pas mécaniquement aux compilations ARM ou MIPS du même code source, ce qui multiplie l'effort de couverture.

1.6.2 Analyse heuristique et émulation

L'analyse heuristique recherche des structures de code ou des séquences d'appels caractéristiques sans se fier à une empreinte exacte. Elle s'appuie souvent sur une émulation partielle destinée à franchir les couches de déballage.

Les techniques anti-analyse décrites plus haut la visent directement. Sur Linux, elle se heurte de surcroît à la diversité des architectures, l'émulation devant être disponible et fidèle pour chacune d'elles.

1.6.3 Analyse comportementale

Les agents de détection observent les actions du programme à l'exécution. Sous Linux, l'outillage contemporain repose massivement sur des sondes eBPF, qui offrent une visibilité fine sur les appels système, les accès fichiers et les connexions réseau, à un coût de performance acceptable.

C'est l'approche la plus robuste face à la variation de forme, puisqu'elle porte sur des invariants d'objectif plutôt que d'implémentation. Ses deux limites ont été exposées plus haut : la source de télémétrie est elle-même attaquable, et un code qui module continûment son comportement en imitant des opérations d'administration légitimes se rapproche progressivement de la distribution normale, ce qui reporte le problème sur la séparation entre dérive légitime et dérive adverse.

1.6.4 Apprentissage automatique

L'apprentissage automatique s'est imposé pour la classification de binaires, et une revue systématique récente en dresse le panorama [5]. Trois difficultés conditionnent la valeur de tout dispositif proposé.

La dérive conceptuelle d'abord. Les performances mesurées en validation se dégradent systématiquement en test sur des échantillons postérieurs, effet du vieillissement des modèles face à l'évolution des familles [19].

Les contraintes opérationnelles ensuite. Un taux de détection annoncé sans le taux de faux positifs associé n'a pas de signification pratique : à l'échelle d'un parc, un taux de faux positifs de 0,1 % produit un volume d'alertes ingérable. Les protocoles d'évaluation récents imposent pour cette raison une mesure à très faible taux de faux positifs et une séparation temporelle des jeux d'entraînement et de test [9].

La disponibilité des données enfin, et c'est ici que le périmètre Linux pèse le plus lourd. Les jeux de données publics de référence portent sur des binaires PE ; les corpus ELF disponibles sont plus petits, moins bien étiquetés et souvent restreints aux familles visant les équipements embarqués [14]. Reproduire sur ELF les protocoles admis dans le monde PE suppose donc de constituer d'abord un corpus. Ce travail n'a pas été mené ici. Le chapitre suivant part d'un seul échantillon, le caractérise, et en tire des règles explicites. Un classifieur entraîné sur ce que j'aurais pu réunir n'aurait pas résisté à l'examen.

1.6.5 Attaques adversariales

Un classifieur est lui-même une surface d'attaque. La génération d'exemples adverses dans l'espace des problèmes obéit à une contrainte absente du domaine de la vision par ordinateur : la perturbation doit produire un fichier valide, exécutable, et dont la sémantique malveillante est préservée [13]. S'y ajoutent l'empoisonnement des données d'entraînement et l'extraction de modèle.

La condition de recevabilité qu'ils fixent vaut au-delà des classifieurs. Une règle écrite à la main tranche elle aussi entre malveillant et bénin, et une règle validée sur le seul échantillon qui l'a produite ne dit rien de ce qu'elle deviendra face à une variante conçue pour la contourner. Les règles du chapitre 2 n'y échappent pas.

1.7 Synthèse et verrou identifié

Les défenses actuelles reposent sur une hypothèse de modélisabilité : que l'adversaire produise des artefacts identifiables, des comportements caractérisables, ou à défaut une distribution assez stable pour qu'un classifieur entraîné hier reste valide demain. L'évasion statique attaque la première hypothèse, les techniques d'exécution la deuxième, et la dérive conceptuelle érode la troisième indépendamment de toute intention adverse.

Le contexte Linux accentue chacun de ces effets. La compilation croisée multiplie les formes d'un même comportement. L'exécution sans fichier supprime l'objet même de l'analyse statique. La télémétrie repose sur une infrastructure noyau unique, dont la compromission prive l'agent de toute perception. Et la rareté des corpus publics prive le chercheur des protocoles

d'évaluation dont dispose son homologue travaillant sur PE.

L'intégration de modèles de langage dans la chaîne offensive n'introduit pas une menace de nature nouvelle. Les échantillons documentés restent expérimentaux, et l'adaptation à l'environnement, souvent présentée comme leur apport spécifique, est déjà attestée sans eux. Ce que ces modèles modifient est le coût de production de la variation, et l'étendue de l'espace des comportements possibles. Une défense dont le coût marginal d'adaptation reste supérieur au coût marginal de mutation de l'adversaire perd du terrain mécaniquement, quelle que soit sa qualité intrinsèque.

Chapitre 2

Étude de cas : analyse de PromptLock

2.1 Présentation du cas

2.1.1 Contexte et découverte

Un mois avant la découverte de PromptLock, le CERT-UA (Computer Emergency Response Team d'Ukraine) signalait *LameHug*, un malware attribué au groupe APT28 et piloté par un modèle de langage, capable de générer à la volée des commandes Windows malveillantes via l'interface de programmation de Hugging Face. *LameHug* reste rudimentaire. Il annonce pourtant la bascule qu'un échantillon autrement plus abouti allait documenter quelques semaines plus tard.

Le 27 août 2025, les chercheurs d'ESET annoncent la découverte de PromptLock, premier *rançongiciel* propulsé par l'intelligence artificielle générative jamais observé. L'échantillon, classifié `Filecoder.PromptLock.A`, a été repéré sur VirusTotal, où deux variantes avaient été téléversées, l'une pour Windows, l'autre pour Linux. Anton Cherepanov, à l'origine de la découverte, résume l'enjeu : en abaissant la barrière technique nécessaire à la conception de menaces sophistiquées, l'intelligence artificielle rend accessible à un attaquant isolé ce qui demandait auparavant une équipe de développeurs expérimentés.

Le 3 septembre 2025, ESET est contacté par les auteurs d'une étude académique intitulée *Ransomware 3.0 : Self-Composing and LLM-Orchestrated*, menée par des doctorants et post-doctorants de la NYU Tandon School of Engineering. Leur prototype ressemble de très près aux échantillons découverts sur VirusTotal. ESET revoit alors son interprétation : PromptLock ne serait pas un outil opérationnel déployé par un groupe cybercriminel, mais le produit de ce projet académique, destiné à illustrer les risques posés par cette nouvelle classe de malwares. L'éditeur maintient la validité de sa découverte technique. Menace active ou non, PromptLock reste le premier cas connu de rançongiciel piloté par un modèle de langage.

PromptLock sera repris en novembre 2025 dans le rapport de synthèse *AI Threat Tracker* du Google Threat Intelligence Group (GTIG), aux côtés de PromptFlux et PromptSteal, rapport qui documente pour sa part des menaces distinctes intégrant des modèles de langage directement dans leur boucle d'exécution.

L'analyse développée dans ce chapitre porte sur l'échantillon documenté par ESET (SHA-256 : 09bf891b7b35b2081d3ebca8de715da07a70151227ab55aec1da26eb769c006f).

2.1.2 Composition et architecture du malware

L'échantillon est un binaire écrit en Go (version `go1.24.5`) et compilé statiquement : il embarque toutes ses dépendances au lieu de les charger depuis le système au moment de l'exécution. La variante étudiée ici vise l'architecture ARM64 sous Linux ; ESET rapporte l'existence d'une variante Windows, signe d'une volonté de rendre l'outil multiplateforme.

Ce qui distingue PromptLock d'un rançongiciel classique, c'est l'absence de toute routine de chiffrement écrite en dur. À la place, le binaire embarque un interpréteur Lua complet, réécrit en Go (le projet `gopher-lua`), avec deux extensions qui lui donnent accès au système de fichiers de la machine infectée et aux opérations de manipulation de bits nécessaires à l'algorithme de chiffrement SPECK en 128 bits.

Le malware repose donc sur un pipeline de génération de code piloté par une intelligence artificielle locale. Des instructions précises, rédigées en langage naturel et codées en dur dans le binaire, partent en requête HTTP vers une interface de programmation compatible avec Ollama, logiciel qui permet d'exécuter des modèles de langage sur sa propre machine. Le modèle interrogé est nommé explicitement : `gpt-oss:20b`, un modèle ouvert de vingt milliards de paramètres. Six tâches s'enchaînent : reconnaissance des fichiers présents sur la machine, écriture d'un programme de chiffrement, rédaction d'une note de rançon. Pour chacune, le modèle renvoie un script Lua que le malware extrait de la réponse et exécute directement en mémoire, sans jamais l'écrire sur le disque. Si l'exécution échoue, le résultat repart vers le modèle sous forme de nouveau prompt, qui corrige et régénère le code, jusqu'à un nombre maximal de tentatives.

La chaîne d'exécution se résume ainsi :

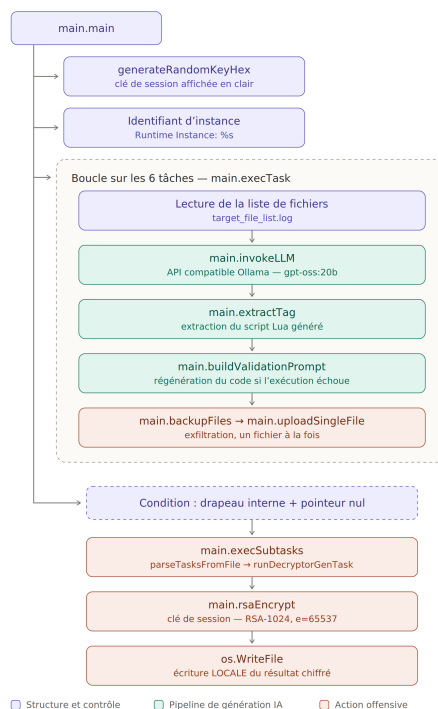


FIGURE 2.1 – Chaîne d'exécution de PromptLock, de `main.main` jusqu'au chiffrement local conditionnel.

Le chemin qui mène au chiffrement local via `main.execSubtasks` n'est pas systématique : il n'est emprunté que si une condition précise est remplie dans le code, un drapeau interne combiné à un test de pointeur nul. La reconnaissance et l'exfiltration se répètent à chaque tâche ; le chiffrement, lui, dépend de cette condition.

2.2 Analyse technique de l'échantillon

2.2.1 Analyse statique

2.2.1.1 Analyse de base

Le triage commence par le calcul des condensats cryptographiques de l'échantillon, ces empreintes numériques qui identifient un fichier de façon unique et permettent de le retrouver dans les bases de menaces connues (VirusTotal, plateformes de renseignement sur les menaces) :

- Condensat MD5 : 806f552041f211a35e434112a0165568
- Condensat SHA-256 : 09bf891b7b35b2081d3ebca8de715da07a70151227ab55aec1da26eb769c006f

Le fichier est un exécutable au format ELF 64 bits pour architecture ARM aarch64. Le format ELF (Executable and Linkable Format) est, sous Linux, l'équivalent du fichier `.exe` de Windows : le format standard des programmes exécutables. La mention « ARM aarch64 » désigne le processeur visé, la famille de puces que l'on trouve dans les smartphones, les objets connectés ou de petits ordinateurs comme le Raspberry Pi, par opposition aux x86_64 des PC de bureau. Le choix n'est pas anodin : il oriente déjà l'analyse vers une cible embarquée, hypothèse confirmée plus loin dans ce chapitre.

Le binaire est ensuite lié statiquement (*statically linked*) : toutes les bibliothèques dont le programme a besoin sont intégrées au fichier exécutable, plutôt que chargées depuis le système au lancement. L'intérêt pour un attaquant est évident. Le malware s'exécute sur n'importe quelle machine Linux ARM64, même dépourvue des bibliothèques habituelles, sans rien installer et donc sans cette trace supplémentaire.

La mention « non-PIE » renvoie à une protection appelée PIE (*Position Independent Executable*), qui charge le programme à une adresse mémoire différente et aléatoire à chaque exécution, ce qui complique l'exploitation de certaines vulnérabilités mémoire. Un binaire non-PIE, comme celui-ci, est toujours chargé à la même adresse. Rien d'offensif là-dedans : c'est le comportement de compilation courant des binaires Go statiquement liés, sans incidence sur le comportement malveillant du programme.

Le binaire a enfin été dépouillé de sa table des symboles (*stripped*). Cette table associe à chaque fonction un nom lisible par un humain, `main.invokeLLM` par exemple ; une fois retirée, l'analyste ne voit plus que des adresses mémoire anonymes et doit reconstruire lui-même la signification de chaque fonction, ce qui ralentit la rétro-ingénierie. La sous-section suivante montre comment contourner l'obstacle en exploitant les métadonnées internes de l'environnement d'exécution Go.

S'y ajoute l'absence complète de section dite « dynamique », normalement présente dans les exécutables qui chargent des bibliothèques partagées externes au lancement : aucune dépendance externe n'est donc chargée dynamiquement, ce que la liaison statique laissait déjà supposer. Ces trois caractéristiques réunies, pas de section dynamique, liaison statique, symboles retirés, sont à elles seules une signature. C'est le profil que produit par défaut le compilateur Go lorsqu'il est configuré pour ne dépendre d'aucune bibliothèque système externe, ce qui a permis aux analystes d'identifier la nature du binaire avant même d'entamer la rétro-ingénierie.

La simple extraction des chaînes de caractères révèle déjà l'essentiel du fonctionnement, alors qu'aucune routine de chiffrement classique n'apparaît en clair dans le code. On y trouve l'adresse de l'interface interrogée par le malware, le nom du modèle `gpt-oss:20b` et des fragments de prompts explicites : la phrase demandant d'implémenter SPECK 128 bits en Lua pur, celle

réclamant que le code généré soit encadré par des balises `<code></code>`. Une adresse Bitcoin figure aussi en clair. C'est l'adresse historique dite *genesis* de Satoshi Nakamoto, valeur de test plutôt que véritable adresse de rançon. Malgré le retrait des symboles, les chemins de compilation d'origine subsistent et pointent vers un dossier `Ransomware3.0/go-llm-ransom`, chez un utilisateur nommé `discover`. Les noms des fichiers journaux produits à l'exécution apparaissent également dans les chaînes extraites, comme `target_file_list.log` ou `note.txt`.

Aucune anomalie d'entropie sur le fichier, ce qui est cohérent avec une compilation Go standard, sans compression ni empaquetage destiné à dissimuler le contenu du binaire.

2.2.1.2 Analyse avancée

La table des symboles a beau avoir été retirée, l'environnement d'exécution Go conserve en interne d'autres métadonnées, qui font correspondre chaque adresse mémoire au nom de la fonction qu'elle représente. En les exploitant avec l'outil GoReSym, j'ai reconstruit l'intégralité de la table des fonctions du programme, ainsi que plusieurs informations injectées lors de la compilation : un nom de compilation qui renvoie explicitement à un environnement de test Raspberry Pi, et une adresse IP privée appartenant à la plage réservée aux réseaux locaux. Un échantillon de développement, donc, pas un outil déployé en conditions réelles. Les dépendances tierces embarquées dans le binaire, l'interpréteur Lua `gopher-lua`, l'extension d'accès au système de fichiers `gopher-lfs` et l'extension d'opérations binaires `gluabit32`, confirment l'architecture décrite dans la section précédente.

La reconstruction du graphe d'appel donne l'organisation suivante :

Un mot de vocabulaire, utile pour la suite du chapitre. « Confirmé » signifie observé directement dans le désassemblage : appel de fonction, référence croisée explicite. « Confirmé par XREF » repose sur une référence croisée (*cross-reference*) entre deux fonctions, la trace que laisse dans le code le fait qu'une fonction en appelle une autre. « Déduit » n'a pas été observé directement, mais découle du comportement environnant : la présence d'un code Lua exploitable juste après la réponse du modèle implique une étape d'extraction, même si l'appel exact n'a pas été isolé avec certitude.

Le décompilateur Ghidra, couplé au module d'analyse spécifique au langage Go, a permis d'aller plus loin dans la compréhension de ces fonctions et de corriger certaines hypothèses posées lors de l'analyse de chaînes. La fonction qui interroge le modèle de langage sérialise le message envoyé au format JSON, puis ajoute à chaque requête un en-tête `Cookie`, construit par concaténation de plusieurs segments, dont une variable partagée avec le mécanisme d'exfiltration décrit plus loin. La fonction qui construit le prompt de validation confirme que les six tâches du malware, avec l'intégralité de leurs instructions système et de leurs critères de réussite, sont codées en dur dans le binaire, et non chargées depuis un fichier de configuration externe comme je l'avais d'abord supposé.

La fonction responsable de l'exécution du code généré crée une machine virtuelle Lua vierge, y charge les deux extensions mentionnées plus haut, puis redéfinit la fonction d'affichage standard de Lua pour capturer sa sortie dans une variable interne au lieu de l'écrire sur la sortie standard. Le code s'exécute ensuite directement depuis la mémoire, sans qu'aucun fichier `.lua` intermédiaire ne soit créé sur le disque. Le point compte pour la suite : il explique pourquoi les approches classiques d'analyse de fichiers passent à côté de cette étape du comportement malveillant.

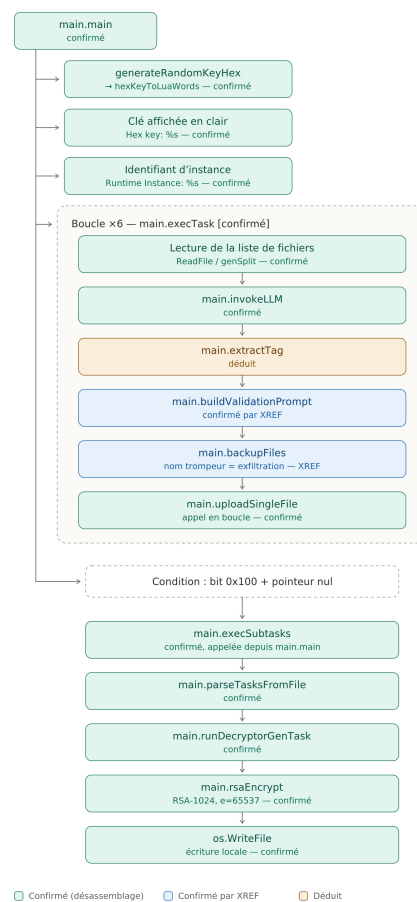


FIGURE 2.2 – Graphe d'appel reconstruit à l'aide de GoReSym et Ghidra, annoté selon le niveau de preuve associé à chaque fonction.

La fonction principale réserve deux surprises. La clé de session utilisée pour le chiffrement, seize octets soit cent vingt-huit bits, est affichée en clair sur la sortie standard au moment de l'exécution : elle est donc directement récupérable dans les journaux, quelle que soit la solidité du chiffrement RSA censé la protéger. Et le champ `session_key`, envoyé lors de l'exfiltration des fichiers, ne contient pas cette clé, mais un simple identifiant d'instance construit à partir du nom de compilation repéré plus haut.

L'exfiltration se produit à l'intérieur de la boucle exécutée pour chacune des six tâches. La fonction `main.backupFiles`, dont le nom laissait supposer une sauvegarde locale des fichiers avant modification, ne sauvegarde rien : elle lit la liste des fichiers ciblés par la tâche en cours et appelle, pour chacun, `main.uploadSingleFile`, qui envoie le fichier vers un serveur distant par une requête HTTP de type formulaire, avec la vérification du certificat vraisemblablement désactivée.

Le chiffrement, lui, n'intervient qu'à la condition évoquée plus haut, ce drapeau interne combiné à un test de pointeur nul, qui déclenche l'appel à `main.execSubtasks`. Cette fonction relit la liste des tâches, puis invoque `main.runDecryptorGenTask`, laquelle appelle `main.rsaEncrypt` pour chiffrer la clé de session. Le chiffrement utilise RSA avec une clé de 1024 bits et l'exposant public standard 65537, taille aujourd'hui insuffisante au regard des standards actuels, qui recommandent 2048 bits au minimum. Contre toute attente pour un rançongiciel, le résultat n'est pas transmis vers l'extérieur mais **écrit localement sur la machine de la victime**, par un appel direct à la fonction d'écriture de fichier de Go. Le scénario se joue donc en deux temps : exfiltration systématique des fichiers ciblés à chaque tâche, puis, sous condition, un chiffrement dont le résultat reste sur place.

La clé publique RSA est reconstruite à partir de grands nombres exprimés en hexadécimal, et non depuis une clé au format habituel. D'où l'absence de chaîne "BEGIN PUBLIC KEY" lors de la recherche de motifs menée pendant l'analyse de base.

Plusieurs indices convergent enfin vers une attribution : les chemins de compilation embarqués, le nom de compilation qui renvoie à un Raspberry Pi, l'adresse IP privée utilisée comme serveur, l'adresse Bitcoin factice. Le faisceau plaide pour un échantillon de recherche ou une preuve de concept, pas pour un outil déployé dans une campagne active. Il rejoint l'hypothèse évoquée plus haut d'un travail issu de la NYU Tandon School of Engineering.

2.2.2 Analyse dynamique

J'ai ensuite confronté les hypothèses de l'analyse statique à une exécution contrôlée de l'échantillon en laboratoire isolé. La plupart ont tenu, quelques-unes ont dû être corrigées.

2.2.2.1 Environnement de laboratoire

L'échantillon étant un binaire ELF ARM64 lié statiquement, j'ai monté deux machines virtuelles VirtualBox, reliées par un segment réseau strictement isolé :

- Kali Linux (x86_64) héberge l'échantillon et les outils de télémétrie. Exécuter un binaire ARM64 sur une machine x86_64 est rendu possible par émulation transparente à l'aide de `qemu-user`, qui enregistre automatiquement l'interpréteur ARM64 auprès du noyau dès qu'un exécutable portant la signature ELF aarch64 est lancé ;

- REMnux héberge INetSim, qui simule les services réseau attendus par le malware (DNS, HTTP/HTTPS) sans jamais l'exposer à une infrastructure réelle.

Les deux machines communiquent par un adaptateur de type *réseau interne*, un segment virtuel totalement coupé de la machine hôte et d'Internet, à la différence du mode *host-only* qui conserve un accès à l'hôte. Le DNS de Kali pointe explicitement vers REMnux, qui répond à toute résolution via INetSim. Avant chaque lancement de l'échantillon, j'ai vérifié l'isolation en constatant qu'une requête vers une adresse publique échoue depuis les deux machines.

L'adresse du serveur de modèle étant codée en dur à la compilation (192.168.1.1, une adresse privée), inutile de modifier le binaire pour intercepter son trafic : un alias IP correspondant sur l'interface réseau de REMnux, une route statique équivalente côté Kali, et tout le trafic destiné au serveur de commande et contrôle est capturé sans toucher à l'échantillon. Plus simple qu'une redirection NAT.

2.2.2.2 Simulation de l'interface de programmation du modèle et premières observations

INetSim répond d'abord à toute requête HTTPS par une réponse JSON statique. J'ai déterminé le format exact attendu par le malware de façon empirique, en lisant ses propres messages d'erreur : un premier essai au format natif d'Ollama échoue au moment du décodage JSON ; un second révèle, via le message "LLM returned no choices", que le format attendu est celui de l'interface OpenAI, avec un champ `choices[0].message.content`, cohérent avec le chemin `/v1/chat/completions` déjà repéré en statique. J'ai ensuite servi à chaque requête une réponse fixe contenant un code Lua de test (`print("test execution successful")`).

Cette première exécution repose sur une réponse figée, incapable donc de déclencher le comportement différencié attendu à chaque tâche. Elle confirme malgré tout une bonne part de ce que l'analyse statique avait déduit :

- les six tâches portent bien les noms `probe`, `scan`, `target`, `extract`, `decide` et `note`, ce qui valide la structure à six étapes déduite du désassemblage de `main.main` ;
- la clé de session s'affiche en clair sur la sortie standard à chaque lancement, au format "Hex key: <32 caractères hexadécimaux>" ;
- l'identifiant d'instance suit le format "Runtime Instance: `rpi_env2_linux_<horodatage>`" : il s'agit bien d'un identifiant de machine, pas d'une clé cryptographique ;
- le moteur Lua embarqué exécute réellement le code renvoyé par le modèle, la sortie capturée par la redéfinition de `print` apparaissant telle quelle dans les journaux ;
- la boucle de validation fonctionne : un échec entraîne une nouvelle itération, avec un prompt de correction qui reprend le code précédemment généré ;
- les fichiers `scan.log`, `payloads.txt` et `note.txt` sont bien produits par le malware en cours d'exécution, tandis que deux fichiers, `sysinfo` et `payloads.txt`, sont en réalité attendus *en entrée* dans le répertoire de travail dès le lancement ;
- la tâche `decide` pilote la sélection d'une sous-tâche via `main.execSubtasks` en traitant la réponse du modèle comme un identifiant (vraisemblablement `encrypt`, `exfiltrate` ou `destroy`) et non comme du code exécutable ; l'échec de recherche de sous-tâche observé avec la réponse de test le confirme.

2.2.2.3 Test avec un modèle de langage réel

La seconde phase remplace la réponse JSON statique par un véritable modèle de langage exécuté localement via Ollama, pour observer le pipeline face à un modèle réel plutôt qu'à une réponse figée. Le binaire référence `gpt-oss:20b`, dont l'exécution demande 16 Go de RAM ou plus ; la machine de laboratoire n'en a que 4. J'ai donc substitué un modèle nettement plus modeste, `qwen2.5:0.5b`. La substitution dégrade la qualité du code généré, mais pas la validité de l'observation : ce que j'observe est le pipeline logiciel, qui dépend du comportement du malware et non de la capacité du modèle interrogé.

Un relais HTTPS écrit pour l'occasion remplace INetSim sur le port 8443. Il reçoit la requête du malware, réécrit le champ `model` pour cibler `qwen2.5:0.5b`, puis la transmet à l'interface OpenAI-compatible native d'Ollama, en local, sans avoir à reformuler la réponse.

Ce second passage donne trois éléments que le premier ne pouvait pas donner :

- un second point de terminaison HTTP, distinct de celui de génération de code : `POST /backup/scripts`. La requête, d'environ 3 ko, ne correspond pas au format JSON attendu par le relais et rejoint le format `multipart/form-data` identifié en désassemblage pour `main.uploadSingleFile`. C'est très vraisemblablement la route d'exfiltration, restée introuvable jusque-là ;
- le délai avant abandon du client HTTP du malware, estimé empiriquement entre 166 et 170 secondes ;
- une génération de code Lua par un modèle réel, observée de bout en bout : le code produit par `qwen2.5:0.5b` pour la tâche `probe` part vers l'interpréteur Lua embarqué, dont l'exécution échoue sur une erreur de syntaxe. L'erreur est correctement réinjectée dans le prompt de correction. La boucle itérative décrite plus haut fonctionne donc en conditions réelles.

J'ai relevé une anomalie. Lors d'une itération où la validation remontait pourtant comme réussie, le pipeline a poursuivi vers une itération supplémentaire au lieu de clore la tâche. Deux explications restent ouvertes : un critère de validation composite non satisfait en parallèle, ou une continuation systématique indépendante du résultat de validation. Trancher demanderait un examen plus poussé du désassemblage de `main.execTask`.

Aucune tâche n'a abouti à un chiffrement de fichier observable au cours de ce test. Le modèle utilisé, quarante fois plus petit que celui visé par le malware, n'a produit aucun code Lua syntaxiquement valide dans le délai imparti sur les prompts les plus complexes, à commencer par l'implémentation de SPECK en Lua pur. La limite est celle du laboratoire, pas celle de la méthode : le malware attend une infrastructure sensiblement plus capable. Un modèle intermédiaire, ou une machine mieux dotée, serait la prochaine étape pour observer un chiffrement SPECK fonctionnel de bout en bout.

2.3 Outils de détection développés

2.3.1 Règle YARA

La détection proposée ici ne vise pas le code Lua généré par le modèle de langage, qui varie par construction d'une exécution à l'autre. Elle vise l'enveloppe qui pilote cette génération,

le binaire Go lui-même, dont l’empreinte reste stable : le nom du modèle utilisé, les fragments de prompts codés en dur, les bibliothèques Lua embarquées. C’est le point que soulève Anton Cherepanov en présentant sa découverte. La charge finale a beau être composée dynamiquement par une intelligence artificielle, le programme qui orchestre cette génération, lui, laisse des marqueurs stables et détectables.

```

1 rule Filecoder_PromptLock_A
2 {
3     meta:
4         description = "Detecte l'enveloppe Go de PromptLock"
5         family      = "Filecoder.PromptLock.A"
6         author      = "GOHAUT Renan"
7         reference    = "ESET - Filecoder.PromptLock.A"
8         hash        = "09bf891b7b35b2081d3ebca8de715da07a70151227ab55aec1da26eb769c006f"
9         date        = "2025"
10
11     strings:
12         $model      = "gpt-oss:20b" ascii
13         $ollama_api  = "/ollama/v1/chat/completions" ascii
14         $prompt_1    = "Implement the SPECK 128bit encryption algorithm" ascii
15         $prompt_2    = "Please provide Lua code wrapped in <code></code> tags" ascii
16         $lua_dep     = "yuin/gopher-lua" ascii
17         $lfs_dep     = "layeh.com/gopher-lfs" ascii
18         $bit32_dep   = "PeerDB-io/gluabit32" ascii
19         $log_1       = "target_file_list.log" ascii
20         $go_magic    = { 47 6F 20 62 75 69 6C 64 69 6E 66 3A }
21
22     condition:
23         uint32(0) == 0x464c457f and
24         $go_magic and
25         $model and
26         ( $ollama_api or 1 of ($prompt_*) ) and
27         2 of ($lua_dep, $lfs_dep, $bit32_dep, $log_1)
28 }
```

Listing 2.1 – Règle YARA ciblant l’enveloppe Go de PromptLock

La règle repose volontairement sur une combinaison de chaînes plutôt que sur un marqueur unique, pour limiter les faux positifs sur d’autres binaires Go qui embarqueraient `gopher-lua` à des fins parfaitement légitimes, l’écriture de scripts applicatifs par exemple. Reste qu’un test contre un corpus de binaires bénins est nécessaire pour mesurer ce taux de faux positifs en conditions réelles.

2.3.2 Règles Sigma

Une règle YARA, aussi solide soit-elle, ne couvre qu’une moitié du problème : elle détecte le binaire orchestrateur au repos, sur disque ou à l’écriture, et ne dit rien du comportement observé une fois le programme lancé. Les résultats de l’analyse dynamique m’ont permis de dériver un second jeu de règles, au format Sigma, qui vise cette fois les traces laissées dans les journaux système, applicatifs et réseau. Ni l’une ni l’autre de ces couches ne suffit seule. Le contenu généré par le modèle étant polymorphe par construction, la détection doit porter sur les invariants du mécanisme, l’enveloppe, et non sur le contenu variable qu’il produit.

- **Création séquentielle des artefacts du pipeline** (confiance moyenne) : détecte l’apparition des fichiers `scan.log`, `payloads.txt`, `note.txt`, `target_file_list.log`, `target_file_info.log` et `target_file_summary.log`. Des noms qu’une application légitime standard a peu de raisons de produire, et que le laboratoire a confirmés comme venant du malware lui-même.
- **Fuite de la clé de session en clair** (confiance haute) : recherche le motif "Hex

`key:"` ou `"Runtime Instance:"` dans la sortie standard collectée des processus. La règle la plus fiable du jeu, cette chaîne exacte ayant peu de chances d'apparaître dans un contexte légitime. Encore faut-il que l'environnement de journalisation capture la sortie standard des processus, ce que beaucoup de configurations ne font pas par défaut.

- **Connexions HTTPS répétées vers un port non standard** (confiance moyenne) : déclenche un seuil au-delà de cinq connexions vers le port 8443 en cinq minutes, cohérent avec le schéma observé de six tâches pouvant chacune itérer jusqu'à huit fois.
- **Corrélation entre lancement d'exécutable et rafale réseau** (confiance moyenne) : corrèle temporellement le démarrage d'un binaire ELF avec l'ouverture rapprochée de plusieurs connexions TLS sortantes. Son efficacité dépend de la prise en charge des corrélations temporelles par le moteur de conversion Sigma employé.
- **Processus enfant curl** (confiance faible) : s'appuie sur le mécanisme de repli identifié dans les prompts codés en dur, qui indique au modèle la conduite à tenir si `curl` n'est pas disponible. Classée volontairement en confiance basse, puisque `curl` est un outil légitime omniprésent : la règle sert de corroboration, pas de détection autonome.

2.3.3 Plugin Volatility3

Dans le cadre d'un projet de groupe mené au sein de nos cours, nous avons développé un plugin pour Volatility 3, orienté vers la détection en mémoire vive des traces d'intelligences artificielles locales. Le projet fait converger deux axes de travail : d'un côté les enseignements dispensés en analyse forensique mémoire, de l'autre les travaux menés pour ce mémoire sur PromptLock, rançongiciel qui exploite des modèles de langage locaux pour générer une partie de son comportement malveillant.

L'outil, nommé LlmFinder, répond à un besoin repéré pendant l'analyse : en réponse à incident, il est difficile de montrer qu'un modèle de langage local a été exécuté sur une machine compromise, que ce soit par Ollama, LM Studio, GPT4All, Jan ou llama.cpp. Ce type d'usage laisse peu de traces disque persistantes, et aucun plugin Volatility 3 existant ne le couvre.

Les menaces qui s'appuient sur des modèles exécutés localement, à l'image de PromptLock, changent la donne pour l'investigation numérique. La logique malveillante n'est plus figée dans un binaire : elle est générée à la volée par un modèle interrogé en local, ce qui réduit fortement le rendement de l'analyse statique et de la détection par signature de fichier.

La mémoire vive devient alors la source de preuve principale. Elle peut conserver, au moment de l'acquisition, ce qui n'existe jamais sous forme persistante sur le disque : le prompt envoyé au modèle, la réponse générée, le point de terminaison local interrogé, le processus qui a servi d'interface. LlmFinder exploite cette fenêtre d'observation pour donner à l'analyste des indices utilisables rapidement.

Le plugin structure ses résultats sur quatre axes, qui suivent la chaîne d'usage d'un modèle local sur un système compromis. Les processus d'abord, avec l'identification de ceux qui correspondent à des applications d'intelligence artificielle locale connues, ce qui établit la présence d'un moteur de génération sur la machine analysée. Les modèles ensuite, par la détection en mémoire de chemins de modèles, qui indiquent lesquels étaient réellement chargés au moment de la compromission. Le réseau, avec les points de terminaison locaux liés aux interfaces de programmation d'intelligence artificielle : des communications internes entre un processus tiers et le moteur de génération signalent souvent un usage programmatique plutôt

qu'interactif. Les prompts enfin, qu'ils aient été soumis par une interface graphique, une interface de programmation locale ou un terminal. C'est la pièce la plus directement exploitable pour reconstituer l'intention et le comportement du code malveillant.

Mises bout à bout, ces quatre familles d'informations reconstituent une chaîne cohérente à partir d'une simple image mémoire : quel processus a sollicité quel modèle, par quel canal, avec quel contenu. L'approche convient bien aux menaces de type PromptLock, où la preuve de compromission ne réside pas dans un artefact statique mais dans l'interaction dynamique entre un binaire et un modèle local.

LlmFinder fournit des indices forensiques, pas des conclusions automatiques. Il ne suffit pas à établir qu'une action malveillante a eu lieu : ses résultats doivent être corrélés avec les autres artefacts, chronologie des événements, processus en cours, fichiers présents, connexions réseau, puis replacés dans le contexte global de l'investigation.

2.3.4 Indices de compromission

Catégorie	Indicateur	Commentaire
Condensat	09bf891b... c006f (SHA-256)	Échantillon ELF ARM64 analysé (ESET)
	806f552041f21... 0165568 (MD5)	Même échantillon
Fichier hôte	target_file_list.log	Liste des fichiers cibles (reconnaissance)
	target_file_info.log	Métadonnées des fichiers ciblés
	target_file_summary.log	Synthèse produite en fin de tâche
	scan.log	Journal de la tâche <code>scan</code>
Fichier hôte	note.txt	Note de rançon générée par le modèle
Chemin de compilation	/home/discover/Ransomware3.0/go-llm-ransom/	Indice d'attribution (preuve de concept de recherche)
Réseau	<serverIP>:8443/ollama/v1/chat/completions	Port non standard (Ollama par défaut : 11434)
Rançon	1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa	Adresse Bitcoin <i>genesis</i> , valeur de remplacement non opérationnelle
Processus / Modèle	gpt-oss:20b	Modèle invoqué via l'interface compatible Ollama
Journaux	"Hex key: %s"	Clé de session affichée en clair sur la sortie standard

TABLE 2.1 – Indices de compromission identifiés lors de l'analyse statique de PromptLock

Le tableau donne raison à la remarque d'Anton Cherepanov citée plus haut. Le code Lua exécuté varie d'une exécution à l'autre, puisque le modèle le régénère à chaque fois. L'enveloppe qui l'orchestre, elle, garde une empreinte statique constante : nom du modèle, adresse de l'interface, artefacts de journalisation, chemins de compilation. C'est cette empreinte qui rend la détection possible, malgré le polymorphisme qu'introduit la génération de code assistée.

L'analyse dynamique a complété ce tableau par des indicateurs réseau que l'analyse statique seule ne donnait pas :

Catégorie	Indicateur	Contexte / confiance
Port	8443/TCP (HTTPS)	Confirmé en statique et en dynamique ; non standard (Ollama par défaut : 11434)
Point de terminaison	POST /ollama/v1/chat/completions	Génération de code, confirmé en dynamique
	POST /backup/scripts	Exfiltration, multipart/form-data, confirmé en dynamique
Champ multipart	session_key	Identifiant d'instance, et non la clé de chiffrement
	file	Contenu du fichier exfiltré
Comportement	Délai d'expiration du client HTTP $\approx$ 166–170 s	Mesuré empiriquement, non extrait du désassemblage
Format requête	JSON compatible OpenAI (model, messages[])	Confirmé en dynamique
Format réponse	choices[0].message.content	Déduit du message d'erreur "LLM returned no choices"

TABLE 2.2 – Indicateurs réseau complémentaires confirmés lors de l'analyse dynamique

L'adresse IP 192.168.1.1 est injectée à la compilation et propre à cet échantillon. Elle ne se généralise pas à d'éventuelles variantes, qui pourraient être compilées avec une autre adresse ou un nom de domaine. Les chemins de terminaison et le format d'échange JSON, en revanche, devraient être plus stables d'une variante à l'autre, puisqu'ils tiennent au code source et non aux paramètres de compilation.

2.3.5 Cartographie MITRE ATT&CK

J'ai classé les techniques observées ou déduites selon le référentiel MITRE ATT&CK, avec un niveau de confiance : confirmée dynamiquement (D), déduite du seul désassemblage statique (S), ou les deux (S+D).

L'absence de technique de persistance traduit une limite de couverture : certaines fonctions n'ont pas été intégralement désassemblées. Ce n'est pas un résultat négatif établi, et le point mériterait une vérification explicite si une couverture exhaustive du binaire devenait nécessaire.

2.3.6 Cartographie MITRE ATLAS

Publié par MITRE sur le même modèle structurel qu'ATT&CK, le référentiel ATLAS (*Adversarial Threat Landscape for Artificial-Intelligence Systems*) documente les surfaces d'attaque propres aux systèmes d'intelligence artificielle, à travers quatorze tactiques et des techniques préfixées AML.T : jeux de données d'entraînement, poids des modèles, interfaces d'inférence, espaces de représentation vectorielle. PromptLock interroge un modèle pendant son exécution même, et pas seulement pendant son développement. Reste à savoir si ATLAS décrit ce comportement plus finement qu'ATT&CK.

Tactique	Technique	Description	Conf.
Execution	T1059	Exécution de code Lua généré dynamiquement via l'interpréteur gopher-lua embarqué	S+D
Defense Evasion	T1027.014	Binaire Go compilé statiquement et partiellement dépouillé de ses symboles ; logique malveillante générée à l'exécution plutôt qu'écrite en dur (<i>Polymorphic Code</i>), ce qui rend la signature de contenu peu pertinente	S
Discovery	T1082	Collecte d'informations système (tâche probe , fichier sysinfo)	S+D
Discovery	T1083	Énumération de fichiers et répertoires via le module lfs exposé à Lua (tâche scan)	S+D
Collection	T1005	Lecture du contenu de fichiers locaux pour analyse (tâches extract / decide)	S
Command and Control	T1071.001	Communication HTTPS applicative vers l'interface du modèle (port 8443, /ollama/v1/chat/completions)	S+D
Exfiltration	T1041	Téléversement des fichiers cibles vers le serveur distant (POST /backup/scripts) avant tout chiffrement, logique de double extorsion	S+D
Impact	T1486	Chiffrement des fichiers cibles via un chiffreur SPECK 128 bits généré dynamiquement en Lua	S ^a
Impact	T1657	Génération d'une note de rançon adaptée au contexte via un prompt dédié	S+D
Persistence	—	Aucun mécanisme de persistance identifié dans les fonctions analysées	— ^b

S : déduit du désassemblage statique • D : confirmé dynamiquement • S+D : les deux.

^a Chaîne reconstituée en statique, non observée fonctionnelle en laboratoire (cf. section 2.2.2.3).

^b Couverture partielle du désassemblage : l'absence n'est pas établie.

TABLE 2.3 – Cartographie des techniques observées ou déduites selon le référentiel MITRE ATT&CK

2.3.6.1 Techniques applicables

Deux techniques ATLAS se rattachent directement au comportement observé lors de l'analyse statique et dynamique (sections 2.2.1 et 2.2.2) :

Tactique	Technique	Application à PromptLock
ML Model Access AML.TA0000	AML.T0040 AI Model Inference API Access	Interrogation systématique de l'interface compatible Ollama (/ollama/v1/chat/completions) pour chacune des six tâches
Execution AML.TA0005	AML.T0050 Command and Scripting Interpreter	Transformation directe de la sortie du modèle en code Lua exécuté par l'interpréteur embarqué

TABLE 2.4 – Techniques MITRE ATLAS applicables à PromptLock

2.3.6.2 Une limite de couverture révélée plutôt que comblée

Au-delà de ces deux correspondances, la cartographie bute sur une limite structurelle du référentiel. La quasi-totalité des techniques ATLAS suppose que le système d'intelligence artificielle est la *cible* de l'attaque : extraction de modèle par requêtes répétées, empoisonnement des données d'entraînement, contournement d'un modèle tiers par injection de prompt. Avec PromptLock, c'est l'inverse. Le modèle n'est pas attaqué, il est *instrumentalisé* comme ressource légitime au service d'un autre objectif malveillant, au même titre qu'un malware s'appuierait sur un service en ligne légitime sans en exploiter la moindre faille.

Ce constat n'est pas isolé. Le rapport *AI Threat Tracker* de novembre 2025 du Google Threat Intelligence Group (GTIG) documente PromptFlux et PromptSteal, deux familles fondées sur le même principe d'appel à un modèle de langage en cours d'exécution, et ne les cartographie pourtant pas sur ATLAS. Il retient des sous-techniques ATT&CK précises : T1027.014 (*Polymorphic Code*) pour la réécriture dynamique de PromptFlux, T1027.016 (*Junk Code Insertion*) pour ses motifs de code leurre. Le choix vient de l'équipe même qui a découvert ces menaces, ce qui vaut confirmation empirique : ATLAS, en l'état, n'offre pas de vocabulaire pour ce schéma précis, le « logiciel malveillant qui utilise une intelligence artificielle comme composant d'exécution », par opposition à son cœur de cible historique, le « système d'intelligence artificielle en tant que victime ».

2.3.6.3 Analyse comparative

Les deux référentiels sont complémentaires plutôt que concurrents : chacun modélise une couche distincte de la même chaîne d'attaque.

2.3.6.4 Conséquence pour la cartographie ATT&CK

D'où une correction apportée à la table 2.3. La technique T1027 (Defense Evasion) gagne à être affinée en T1027.014 (*Polymorphic Code*) : le comportement observé, du code malveillant généré à l'exécution plutôt qu'écrit en dur, correspond à cette sous-technique, et c'est la classification que le GTIG a retenue pour PromptFlux.

Critère	MITRE ATT&CK	MITRE ATLAS
Objet modélisé	Le comportement de l'attaquant, indépendamment de l'outillage utilisé	La surface d'attaque propre aux systèmes d'intelligence artificielle
Pertinence pour PromptLock	Élevée : couvre l'intégralité de la chaîne (table 2.3)	Partielle : seules deux techniques s'appliquent (accès à l'interface d'inférence, exécution de commandes)
Cas d'usage idéal	Malware générique, y compris à composant d'intelligence artificielle	Attaque <i>contre</i> un modèle (extraction, empoisonnement, contournement des garde-fous)
Limite observée	Aucune sous-technique dédiée à la génération de code par modèle de langage au moment de la rédaction	Absence de technique couvrant l'intelligence artificielle comme <i>outil</i> plutôt que comme <i>cible</i>

TABLE 2.5 – Analyse comparative ATT&CK / ATLAS appliquée à PromptLock

La cartographie ATLAS n'enrichit donc pas la table d'indicateurs. Elle produit autre chose, un résultat méthodologique : la mesure, à partir d'un cas concret, d'un angle mort du référentiel face à cette classe émergente de menaces. Seule une évolution d'ATLAS, ou de nouvelles sous-techniques ATT&CK consacrées à la génération de code par modèle de langage, le comblerait.

Conclusion

L'échantillon étudié appartient à une classe de menaces encore peu documentée : les codes malveillants qui interrogent un modèle de langage pendant leur propre exécution. Le caractère a demandé de reconstruire un pipeline dont presque rien ne se répétait d'une exécution à l'autre. L'analyse statique a livré six tâches codées en dur, une clé de session en clair, un chemin de compilation oublié et les prompts système embarqués dans le binaire ; l'analyse dynamique a confirmé les noms de tâches, les artefacts produits, l'endpoint d'exfiltration et un délai d'expiration côté client d'environ 170 secondes. J'en ai tiré un jeu de règles YARA, cinq règles Sigma et les fondations d'un plugin Volatility.

Chez PromptLock, la génération varie mais l'orchestrateur non. Le code Lua diffère à chaque exécution, la note de rançon aussi, le contenu chiffré ne se répète jamais. L'enveloppe qui pilote l'ensemble, elle, ne bouge pas : imports Go, prompts système, noms de tâches, chemin de compilation, format des échanges avec l'API. La détection reste donc possible, à condition de viser la structure qui produit le contenu plutôt que le contenu lui-même.

Cette stabilité n'a rien d'acquis. PromptFlux, documenté par le GTIG en novembre 2025, vise l'inverse : faire réécrire son propre code source par le modèle. La tentative n'aboutit pas, mais l'échantillon analysé ici en est sans doute l'état antérieur.

Ni ATT&CK ni ATLAS ne décrivent l'usage d'un modèle comme outil au sein d'un code malveillant, par opposition à l'attaque menée contre un système d'IA. Le GTIG s'est lui-même rabattu sur des sous-techniques existantes pour cartographier PromptFlux. Tant que ce vocabulaire manque, deux analystes qui observent le même comportement le nomment autrement, et le renseignement circule mal.

Un seul échantillon, donc, et probablement un prototype de recherche : adresse Bitcoin de test, nom de compilation conservé. Les règles n'ont pas été confrontées à des variantes modifiées pour les contourner. Leur taux de faux positifs reste à mesurer, et sans cette mesure l'équipe ne reprendra rien : une règle qui bruite sur un parc entier finit désactivée, juste ou pas.

Un binaire qui chargerait ses instructions depuis un serveur, ou les dériverait d'un modèle local, ne laisserait plus aucune prise à YARA et reporterait l'effort sur la télémétrie comportementale. S'y ajoute la baisse du coût d'entrée : le marché des outils offensifs assistés par IA s'est structuré en 2025, et ce qui demande aujourd'hui de savoir intégrer une API dans un binaire Go sera bientôt vendu clé en main.

Les règles écrites ici vieilliront, plus vite que celles d'un rançongiciel classique. La question qui les a produites se repose à chaque nouvel échantillon : qu'est-ce qui ne peut pas varier sans que la menace cesse de fonctionner ?

Bibliographie

- [1] ANSSI. *Panorama de la cybermenace 2025*. Rapport CERTFR-2026-CTI-002. CERT-FR, Agence nationale de la sécurité des systèmes d'information. Mars 2026. URL : <https://www.cert.ssi.gouv.fr/cti/CERTFR-2026-CTI-002/> (visité le 13/08/2026).
- [2] Vesselin BONTCHEV. « Current Status of the CARO Malware Naming Scheme ». In : *Proceedings of the 15th Virus Bulletin International Conference*. Dublin, Irlande, oct. 2005. URL : <https://bontchev.nlc.v.bas.bg/papers/naming.html> (visité le 13/08/2026).
- [3] Christian COLLBERG, Clark THOMBORSON et Douglas LOW. *A Taxonomy of Obfuscating Transformations*. Technical Report 148. Department of Computer Science, University of Auckland, juill. 1997.
- [4] NYU Tandon School of ENGINEERING. *Large language models can execute complete ransomware attacks autonomously*. 27 août 2025. URL : <https://engineering.nyu.edu/news/large-language-models-can-execute-complete-ransomware-attacks-autonomously-nyu-tdan-research> (visité le 13/08/2026).
- [5] Matthew G. GABER, Mohiuddin AHMED et Helge JANICKE. « Malware Detection with Artificial Intelligence : A Systematic Literature Review ». In : *ACM Computing Surveys* 56.6 (2024), p. 1-33. DOI : [10.1145/3638552](https://doi.org/10.1145/3638552).
- [6] AV-TEST GMBH. *Statistiques sur les logiciels malveillants*. AV-TEST Institute. 2026. URL : <https://www.av-test.org/fr/statistiques/logiciels-malveillants/> (visité le 13/08/2026).
- [7] Google Threat Intelligence GROUP. *GTIG AI Threat Tracker : Advances in Threat Actor Usage of AI Tools*. Google Cloud. 5 nov. 2025. URL : <https://cloud.google.com/blog/topics/threat-intelligence/threat-actor-usage-of-ai-tools> (visité le 13/08/2026).
- [8] Google Threat Intelligence GROUP. *GTIG AI Threat Tracker : Adversaries Leverage AI for Vulnerability Exploitation, Augmented Operations, and Initial Access*. Google Cloud. 11 mai 2026. URL : <https://cloud.google.com/blog/topics/threat-intelligence/ai-vulnerability-exploitation-initial-access> (visité le 13/08/2026).
- [9] Robert J. JOYCE, Gideon MILLER, Phil ROTH, Richard ZAK, Elliott ZARESKY-WILLIAMS, Hyrum ANDERSON, Edward RAFF et James HOLT. « EMBER2024 : A Benchmark Dataset for Holistic Evaluation of Malware Classifiers ». In : *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2025. DOI : [10.1145/3711896.3737431](https://doi.org/10.1145/3711896.3737431).
- [10] Elastic Security LABS. *Hooked on Linux : Rootkit Taxonomy, Hooking Techniques and Tradecraft*. Partie 1 sur 2. Elastic. 5 mars 2026. URL : <https://www.elastic.co/security-labs/linux-rootkits-1-hooked-on-linux> (visité le 13/08/2026).
- [11] MICROSOFT. *How Microsoft names malware*. Microsoft Learn. 2025. URL : <https://learn.microsoft.com/fr-fr/unified-secops/malware-naming> (visité le 13/08/2026).

- [12] MITRE. *MITRE ATT&CK Enterprise Matrix*. Version à préciser. MITRE Corporation. 2026. URL : <https://attack.mitre.org/> (visité le 13/08/2026).
- [13] Fabio PIERAZZI, Feargus PENDLEBURY, Jacopo CORTELLAZZI et Lorenzo CAVALLARO. « Intriguing Properties of Adversarial ML Attacks in the Problem Space ». In : *2020 IEEE Symposium on Security and Privacy*. San Francisco, CA : IEEE, 2020, p. 1332-1349. DOI : [10.1109/SP40000.2020.00073](https://doi.org/10.1109/SP40000.2020.00073).
- [14] Anandharaju Durai RAJU, Ibrahim Y. ABUALHAOL, Ronnie Salvador GIAGONE, Yang ZHOU et Shengqiang HUANG. « A Survey on Cross-Architectural IoT Malware Threat Hunting ». In : *IEEE Access* 9 (2021), p. 91686-91709. arXiv : [2306.07989](https://arxiv.org/abs/2306.07989).
- [15] ESET RESEARCH. *First known AI-powered ransomware uncovered by ESET Research*. Mise à jour du 3 septembre 2025. WeLiveSecurity. 27 août 2025. URL : <https://www.welivesecurity.com/en/ransomware/first-known-ai-powered-ransomware-uncovered-eset-research/> (visité le 13/08/2026).
- [16] SYNACKTIV. *LinkPro : eBPF rootkit analysis*. Synacktiv. 14 oct. 2025. URL : <https://www.synacktiv.com/en/publications/linkpro-ebpf-rootkit-analysis> (visité le 13/08/2026).
- [17] Péter SZÖR. *The Art of Computer Virus Research and Defense*. Addison-Wesley, 2005. ISBN : 978-0321304544.
- [18] Xabier UGARTE-PEDRERO, Davide BALZAROTTI, Igor SANTOS et Pablo G. BRINGAS. « SoK : Deep Packer Inspection : A Longitudinal Study of the Complexity of Run-Time Packers ». In : *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, p. 659-673. DOI : [10.1109/SP.2015.46](https://doi.org/10.1109/SP.2015.46).
- [19] Limin YANG, Arridhana CIPTADI, Ihar LAZIUK, Ali AHMADZADEH et Gang WANG. « BODMAS : An Open Dataset for Learning based Temporal Analysis of PE Malware ». In : *4th Deep Learning and Security Workshop (DLS), co-localisé avec IEEE Symposium on Security and Privacy*. 2021. URL : <https://whyisyoung.github.io/BODMAS/> (visité le 13/08/2026).

Chapitre A

Annexes

A.1 Captures d'écran

A.2 Formules chimiques

A.3 Extraits de code source

Table des matières

Remerciements

Liste des tableaux

Table des figures

Introduction	1
1 État de l'art de la menace logicielle et mécanismes d'évasion	2
Périmètre	2
1.1 Définition et taxonomie du malware moderne	2
1.1.1 Nomenclature et standards d'identification	3
Les limites du schéma.	3
1.1.2 Tactique, technique, vecteur, capacité et charge utile	4
1.2 Classification par objectifs opérationnels	4
1.2.1 L'écosystème économique	5
1.3 Évasion statique : rendre le binaire illisible	6
1.3.1 Réduction de la surface d'analyse	6
1.3.2 Empaquetage	6
1.3.3 Polymorphisme, métamorphisme et virtualisation	6
1.4 Évasion à l'exécution : échapper à l'observation	7
1.4.1 Anti-analyse et conditionnement de l'environnement	7
1.4.2 Exécution sans fichier	8
1.4.3 Dissimulation par rootkit : trois couches	8
Un cas instructif.	8
1.4.4 Neutralisation de la télémétrie	9
1.4.5 Persistance et mimétisme	9
1.4.6 Commande, contrôle et exfiltration	9
1.4.7 Contrepoint : les spécificités Windows	9
1.5 Intégration de modèles de langage dans la chaîne offensive	10
1.5.1 Les cas documentés	10
1.5.2 Ce que ces cas établissent, et ce qu'ils n'établissent pas	10
1.5.3 L'injection de prompt comme technique d'évasion	11
1.5.4 L'adaptation contextuelle : une capacité déjà présente	11
1.6 État de l'art de la détection et de ses limites	12
1.6.1 Détection statique par indicateurs	12
1.6.2 Analyse heuristique et émulation	12
1.6.3 Analyse comportementale	12
1.6.4 Apprentissage automatique	13
1.6.5 Attaques adversariales	13
1.7 Synthèse et verrou identifié	13
2 Étude de cas : analyse de PromptLock	15
2.1 Présentation du cas	15
2.1.1 Contexte et découverte	15
2.1.2 Composition et architecture du malware	15
2.2 Analyse technique de l'échantillon	16
2.2.1 Analyse statique	17
2.2.1.1 Analyse de base	17

2.2.1.2	Analyse avancée	18
2.2.2	Analyse dynamique	20
2.2.2.1	Environnement de laboratoire	20
2.2.2.2	Simulation de l'interface de programmation du modèle et premières observations	21
2.2.2.3	Test avec un modèle de langage réel	22
2.3	Outils de détection développés	22
2.3.1	Règle YARA	22
2.3.2	Règles Sigma	23
2.3.3	Plugin Volatility3	24
2.3.4	Indices de compromission	25
2.3.5	Cartographie MITRE ATT&CK	26
2.3.6	Cartographie MITRE ATLAS	26
2.3.6.1	Techniques applicables	28
2.3.6.2	Une limite de couverture révélée plutôt que comblée	28
2.3.6.3	Analyse comparative	28
2.3.6.4	Conséquence pour la cartographie ATT&CK	28
Conclusion		30
Bibliographie		
A Annexes		I
A.1	Captures d'écran	I
A.2	Formules chimiques	I
A.3	Extraits de code source	I

